

PORTADA

Técnicas de Programación

Semana 1 — Operadores Aritméticos y Ciclos

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



Variables, literales y operadores

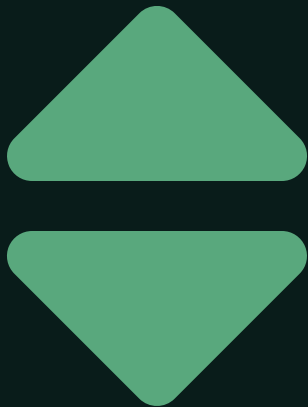
Recordemos: una **variable** es una posición de memoria en la que se puede almacenar un valor que puede cambiar durante la ejecución de un programa. Un **literal** es la representación de un valor en el código fuente del programa.

Operador	Significado	Ejemplo	Resultado
+	Suma	a + b	Suma de a y b
-	Resta	a - b	Resta de a y b
*	Multiplicación	a * b	Producto de a por b
/	División	a / b	Cociente de a entre b
%	Residuo	a % b	Residuo de a entre b



Jerarquía de operadores aritméticos

Operador	Precedencia
<code>()</code>	Se evalúa en primer lugar.
<code>* / %</code>	Se evalúa en segundo lugar.
<code>+ -</code>	Se evalúa al último.



EXPRESIONES ARITMÉTICAS EN JAVA

Ejercicio: traducir expresiones algebraicas a Java

Escribe las siguientes expresiones algebraicas en Java:

- **a.** $e = a^2 + 3bc + 2$
- **b.** $z = (a + b + 2) / (a^2 + 1) + 2ab$
- **c.** $\text{prom} = (n1 + n2 + n3 + n4) / 4$



OPERADORES LÓGICOS

Operador Y (&&)

	true	false
true	true	false
false	false	false

```
int a = 5;  
int b = -1;  
(a > b) && (b < 0); // true
```

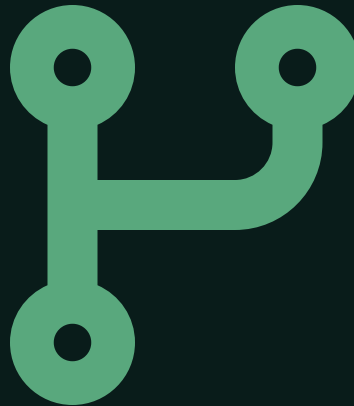


OPERADORES LÓGICOS

Operador O (||)

	true	false
true	true	true
false	true	false

```
int a = 5;  
int b = -1;  
(b > 0) || (a < b); // true
```



OPERADORES LÓGICOS

Negación (!)

El operador de negación evalúa un único operando lógico y devuelve como resultado su valor invertido. Es decir, si el operando tiene el valor `true`, este operador devolverá `false`, y viceversa.

```
int a = 9;  
!(a == 9); // false
```



OPERADORES LÓGICOS · BONIFICACIÓN

¿Cuál es el resultado de este programa?

```
String name = null;  
if (name != null & name.contains("juan")) {  
    System.out.println("Juan is contained in the string");  
} else {  
    System.out.println("Juan is not contained in the string");  
}
```

Pista: a diferencia de `&&`, el operador `&` no aplica cortocircuito: evalúa ambos lados aunque el primero ya sea `false`. Aquí eso provoca una `NullPointerException` al intentar llamar `.contains()` sobre `null`.

PRÁCTICA

Ejercicios

1. Declara dos variables numéricas (con el valor que quieras) y muestra por consola la suma, resta, multiplicación, división y módulo (residuo de la división).
2. Declara dos variables numéricas e indica cuál es mayor de las dos. Si son iguales, indícalo también. Cambia los valores para comprobar que funciona.
3. Declara un String con tu nombre y muestra un mensaje de bienvenida por consola. Por ejemplo: si ingreso "Fernando", obtengo "Bienvenido Fernando".
4. Modifica la aplicación anterior para que pida por teclado el nombre que queremos ingresar.
5. Crea una aplicación que calcule el área de un círculo ($\pi \cdot R^2$). El radio se pedirá por teclado.

ACUMULADOR Y CONTADOR

Operadores de incremento y decremento

```
int count = 1;
count = count + 1;    // Operación: incrementar en 1
count++;              // Expresión en Java para incrementar en 1

int acum = 1;
acum = acum + (acum * 2); // Operación: acumular un valor en la misma variable
acum += (acum * 2);      // Expresión en Java para acumular
```

Nombre	Operador	Variante	Uso	Explicación
Incremento	++	Prefijo	++a	Incrementa a en 1 y luego usa el nuevo valor.
		Postfijo	a++	Usa el valor actual de a y luego lo incrementa en 1.
Decremento	--	Prefijo	--a	Decrementa a en 1 y luego usa el nuevo valor.
		Postfijo	a--	Usa el valor actual de a y luego lo decrementa en 1.

CICLOS EN JAVA

Ciclo "For"

El ciclo `for` permite ejecutar una sentencia de Java múltiples veces, y se usa principalmente cuando se conoce el número total de repeticiones antes de la ejecución.

Sintaxis: el ciclo `for` tiene tres partes: la primera itera la posición del arreglo, la segunda valida cuándo termina el ciclo, y la tercera incrementa la iteración.

```
for (int i = 0; i < arrayList.length; i++) {  
    System.out.println(arrayList[i]);  
}
```



CICLOS EN JAVA

Ciclo "While"

El ciclo `while` permite ejecutar una sentencia de Java múltiples veces hasta que se cumpla la condición. Tiene dos partes: primero la condición, antes de entrar al ciclo, y luego la iteración durante el ciclo.

```
var i = 0;
while (i < arrayList.length) {
    System.out.println(arrayList[i]);
    i++;
}
```



CICLOS EN JAVA

Ciclo "Do while"

El ciclo `do while` permite ejecutar una sentencia de Java múltiples veces hasta que se cumpla la condición. Tiene dos partes: primero la iteración durante el ciclo, y luego la condición, después de entrar al ciclo.

```
var i = 0;  
do {  
    System.out.println(arrayList[i]);  
    i++;  
} while (i < arrayList.length);
```



CICLOS EN JAVA

Ciclo "ForEach"

El uso de `forEach` nos permite recorrer la lista de elementos de forma más compacta. Solo hace falta declarar la variable con el tipo del objeto que se va a iterar.

```
for (int number : arrayList) {  
    System.out.println(number);  
}
```



CONVERSIÓN Y CASTING

Compatibilidad de tipos y parsing de String

Tipo origen	Tipo destino
byte	double, float, long, int, char, short
short	double, float, long, int
char	double, float, long, int
int	double, float, long
long	double, float
float	double

```
String s = "200";
int    a = Integer.parseInt(s);
double b = Double.parseDouble(s);
float  c = Float.parseFloat(s);
```

CONVERSIÓN Y CASTING

Casting implícito y explícito

Widening casting (ampliación): se hace automáticamente al pasar de un tipo de menor tamaño a uno de mayor tamaño.

```
int myInt = 9;
double myDouble = myInt; // Conversión automática: int a double

System.out.println(myInt);    // Imprime 9
System.out.println(myDouble); // Imprime 9.0
```

Narrowing casting (reducción): debe hacerse manualmente colocando el tipo entre paréntesis delante del valor.

```
double myDouble = 9.78d;
int myInt = (int) myDouble; // Conversión manual: double a int

System.out.println(myDouble); // Imprime 9.78
System.out.println(myInt);    // Imprime 9
```


EJERCICIO

Adivina el número

Usa `Math.random()` para elegir un número entre 1 y 100.

Crea una aplicación que permita adivinar un número: la aplicación genera un número aleatorio entre 1 y 100 y luego pide números, respondiendo si el número a adivinar es mayor o menor que el ingresado, además de los intentos restantes (tienes 10 intentos para adivinarlo).

El programa termina cuando se adivina el número (indicando en cuántos intentos se logró); si se llega al límite de intentos, muestra el número que se había generado.

PARA LLEVAR

Ideas clave de la sesión

- La jerarquía de operadores decide el orden de evaluación: paréntesis, luego `* / %`, y al final `+ -`.
- `&&` y `||` aplican cortocircuito; `&` y `|` no — evalúan siempre ambos lados, lo que puede causar errores como el de la bonificación.
- Elige `for` cuando conoces el número de repeticiones, `while / do while` cuando depende de una condición, y `forEach` para recorrer colecciones.
- El widening casting es automático; el narrowing casting siempre debe hacerse explícito con paréntesis.

